

Assignment-2

Data Visualization and Analytics Project: Social Media Trends Analytics Dashboard

Project Link: <https://github.com/akshatripathi1/Social-Media-Trends-Dashboard>

Dataset: Custom Social Media Trends Dataset (2000 records, 12 columns)

Project Overview

This project presents a **multi-page interactive dashboard** developed using Python, Tkinter, Pandas, and Matplotlib to analyze **social media trends and engagement metrics** across different platforms.

The system transforms raw social media data into a **visual analytics platform** that helps users understand platform performance, content engagement patterns, and trending hashtags.

The dashboard is designed to provide insights into:

- Social media engagement trends across Twitter, Instagram, YouTube, Facebook, and TikTok
- Hashtag performance and trending analysis across 10 content categories
- Platform-wise reach comparison (Views, Likes, Comments, Shares)
- Content category engagement patterns (Entertainment, Sports, Technology, etc.)
- Metric correlation analysis (Posts vs Likes vs Views vs Engagement Rate)
- Platform × Category engagement heatmap for identifying content sweet spots

The interface follows a **clean modern UI design** with a midnight purple theme and color-coded platform indicators for better visual understanding.

Dataset Overview

The dataset contains **2000 records of social media trend data** simulating realistic engagement patterns across multiple platforms and content categories.

It includes:

- Platform (Twitter, Instagram, YouTube, Facebook, TikTok)
- Hashtag name (60 unique hashtags)
- Content Category (Entertainment, Sports, Politics, Technology, Fashion, Gaming, Food, Travel, Education, Health)
- Country of origin (11 countries, India-biased)
- Date and Month

- Metrics:
 - o Posts count
 - o Likes
 - o Views
 - o Comments
 - o Shares
 - o Engagement Rate (%)

This dataset is used to analyze social media trends and visualize engagement patterns effectively.

Tech Stack

Python → Programming language

Tkinter → GUI framework for dashboard

Pandas → Data cleaning and analysis

Matplotlib → Data visualization (charts & graphs)

NumPy → Numerical computation and data processing

Program Code

```

"""
Social Media Trends Analytics Dashboard
DVA Assignment 2 | Akshat Tripathi | 04914202023 | IPU BCA 6th Sem
This module contains the complete implementation of the Social Media Trends
Analytics Dashboard. It is designed to provide comprehensive visualization
and analytics capabilities for social media data across various platforms.
The architecture follows a procedural pattern heavily utilizing Tkinter
for the graphical user interface and Matplotlib for the data visualizations.
Data is manipulated and aggregated using the Pandas library to ensure
high performance on datasets with thousands of records.
Features:
- Platform Reach Analysis
- Category Engagement Trends
- Hashtag Rankings (Top & Bottom performers)
- Metric Correlation Heatmaps
- Summary KPIs and distribution breakdowns
Author: Akshat Tripathi
Enrollment: 04914202023
Class: IPU BCA 6th Semester
"""

import tkinter as tk
from tkinter import ttk
import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from matplotlib.figure import Figure
from matplotlib.gridspec import GridSpec
import os
import sys
import logging
from datetime import datetime

# =====
# LOGGING CONFIGURATION
# =====
# We set up logging to ensure that all events within the dashboard are tracked.
# This helps in debugging data loading issues and tracking UI interactions.
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger('SMDashboard')

# =====
# CONSTANTS AND THEME CONFIGURATION
# =====
# MIDNIGHT PURPLE THEME
# The color palette has been carefully selected to provide high contrast
# and a modern, sleek appearance reminiscent of popular analytics tools.
BG = "#13111C"          # Main background color
CARD = "#1E1B2E"         # Card and panel background
CARD2 = "#252238"        # Alternate card background for tables
FG = "#F0E6FF"           # Primary text color (Foreground)
ACCENT = "#B983FF"       # Primary accent color (Purple)
ACCENT2 = "#D4BBFF"      # Secondary accent color (Lighter Purple)
GOOD = "#00E396"         # Success/Positive indicator color (Green)
WARN = "#FF6178"         # Warning/Negative indicator color (Red)
MID = "#FEB019"          # Neutral/Midpoint indicator color (Orange)
MUTED = "#8A85A0"        # Muted text color for subtitles and labels
BORDER = "#362F50"       # Border color for cards and widgets
# Brand colors corresponding to popular social media platforms
PLATFORM_COLORS = {

```

```

    "Twitter": "#1DA1F2",
    "Instagram": "#E4405F",
    "YouTube": "#FF0000",
    "Facebook": "#1877F2",
    "TikTok": "#25F4EE"
}
# Distinct colors mapped to various content categories
CATEGORY_COLORS = {
    "Entertainment": "#FF6B6B",
    "Sports": "#4ECDC4",
    "Politics": "#FFE66D",
    "Technology": "#A29BFE",
    "Fashion": "#FD79A8",
    "Gaming": "#6C5CE7",
    "Food": "#FDCB6E",
    "Travel": "#00B894",
    "Education": "#74B9FF",
    "Health": "#55EFC4"
}

# Matplotlib styling updates
# This block globally updates matplotlib's parameters to match the Tkinter theme.
# Doing this globally ensures that all figures created will inherently follow
# the midnight purple styling without requiring manual color assignments on every axis.
mpl.rcParams.update({
    "font.family": "sans-serif",
    "font.sans-serif": ["Segoe UI", "DejaVu Sans", "Arial"],
    "font.size": 11,
    "axes.titlesize": 13,
    "axes.labelsize": 11,
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "legend.fontsize": 10,
    "figure.facecolor": BG,
    "axes.facecolor": CARD,
    "axes.edgecolor": BORDER,
    "axes.labelcolor": MUTED,
    "xtick.color": FG,
    "ytick.color": FG,
    "text.color": FG,
    "grid.color": BORDER,
    "grid.linewidth": 0.5
})

# Display constants
DPI = 100
FILE_PATH = "Social_Media_Trends_India.csv"
FONT_TABLE = ("Segoe UI", 10)
FONT_THEAD = ("Segoe UI", 10, "bold")
# =====
# DATA HANDLING AND VALIDATION UTILITIES
# =====
class DataValidationError(Exception):
    """Custom exception raised when data validation fails."""
    pass
def validate_dataset(df):
    """
    Validates the structure and content of the loaded dataframe.
    Args:
        df (pd.DataFrame): The dataframe to validate.
    Raises:
        DataValidationError: If critical columns are missing.
    """
    required_cols = {'platform', 'category', 'hashtag', 'views',
                     'likes', 'comments', 'shares', 'posts', 'engagement_rate'}
    missing = required_cols - set(df.columns)

```

```

    if missing:
        msg = f"Dataset is missing required columns: {missing}"
        raise DataValidationError(msg)
    return True
def load_data():
    """
    Load the dataset from the specified CSV file.
    Performs data cleaning, normalizes column names, and validates content.
    Returns:
        pd.DataFrame: The loaded and cleaned dataset ready for visualization.
    """
    try:
        if not os.path.exists(FILE_PATH):
            df = pd.read_csv(FILE_PATH)
            # Normalize column names by stripping whitespace and converting to lowercase
            df.columns = [c.strip().lower() for c in df.columns]
            # Validate dataset structure
            validate_dataset(df)
            return df
    except Exception as e:
        # Return an empty dataframe with expected columns as fallback to prevent UI crashes
        return pd.DataFrame(columns=[
            'platform', 'category', 'hashtag', 'views',
            'likes', 'comments', 'shares', 'posts', 'engagement_rate',
            'country', 'date'
        ])
# =====
# FORMATTING UTILITIES
# =====
def fmt_num(n):
    """
    Format a raw number into a human-readable string with K or M suffixes.
    This is extremely useful for displaying large social media metrics
    like views and likes without cluttering the UI.
    Args:
        n (float/int/str): The numerical value to format.
    Returns:
        str: The formatted string (e.g., '1.5M', '45.2K', '800')
    """
    try:
        n = float(n)
        if np.isnan(n):
            return "0"
        if n >= 1e6:
            return f"{n/1e6:.1f}M"
        if n >= 1e3:
            return f"{n/1e3:.1f}K"
        return str(int(n))
    except (ValueError, TypeError):
        return "0"
def trunc(s, n=14):
    """
    Truncate a string to a specific maximum length and append an ellipsis.
    Used primarily for tick labels in charts to prevent overlapping text.
    Args:
        s (str): The string to truncate.
        n (int): Maximum allowed length before truncation.
    Returns:
        str: The potentially truncated string.
    """
    s = str(s)
    if len(s) <= n:
        return s
    return s[:n-1] + "..."

```

```

# =====
# MATPLOTLIB UI INTEGRATION HELPER FUNCTIONS
# =====
def style_ax(ax, title="", xlabel="", ylabel=""):
    """
    Apply standard theming and styling to a matplotlib axes object.

    Args:
        ax (matplotlib.axes.Axes): The axes object to style.
        title (str): Optional title to display above the axes.
        xlabel (str): Optional x-axis label.
        ylabel (str): Optional y-axis label.
    """
    ax.set_facecolor(CARD)

    # Style the bounding box (spines)
    for sp in ax.spines.values():
        sp.set_color(BORDER)

    # Style the ticks
    ax.tick_params(axis="x", colors=FG)
    ax.tick_params(axis="y", colors=FG)

    # Set titles and labels with specific thematic colors
    if title:
        ax.set_title(title, color=ACCENT2, fontsize=12, fontweight="bold", pad=8)
    if xlabel:
        ax.set_xlabel(xlabel, color=MUTED)
    if ylabel:
        ax.set_ylabel(ylabel, color=MUTED)

def fix_yticks(ax, fs=10):
    """
    Standardize the y-ticks on a horizontal bar chart.
    Ensures that labels are horizontal (not rotated) and sized correctly.
    """
    ax.tick_params(axis="y", labelrotation=0, labelsize=fs, colors=FG)

def hbar(ax, labels, values, colors, fs=10, bh=0.65, vlabels=True):
    """
    Create a styled horizontal bar chart with optional value annotations.

    Args:
        ax (matplotlib.axes.Axes): Matplotlib axes object.
        labels (list): The categorical y-axis labels.
        values (list): The numerical values determining bar length.
        colors (list/str): Colors mapping to the bars.
        fs (int): Font size for annotations and ticks.
        bh (float): Height/thickness of the bars (0.0 to 1.0).
        vlabels (bool): Whether to auto-annotate the bars with their values.

    Returns:
        matplotlib.container.BarContainer: The created bar plot object.
    """
    bars = ax.barh(labels, values, color=colors, height=bh, edgecolor=BORDER, linewidth=0.5)

    if vlabels:
        # Add numeric labels at the end of each bar
        ax.bar_label(bars, fmt="%.1f", color=FG, fontsize=fs-1, padding=3)

    fix_yticks(ax, fs)
    return bars

```

```

def embed(fig, parent):
    """
    Embed a matplotlib figure into a Tkinter parent frame container.
    This links the matplotlib rendering engine with the Tkinter GUI.

    Args:
        fig (matplotlib.figure.Figure): The figure to render.
        parent (tk.Widget): The tkinter frame/widget to host the canvas.
    """
    c = FigureCanvasTkAgg(fig, master=parent)
    c.draw()
    # Pack the canvas widget to fill the entire parent container
    c.get_tk_widget().pack(fill="both", expand=True)

def cell_tc(cmap, nv):
    """
    Calculate the appropriate text color (black or white) based on the
    luminance of the background color generated by a colormap.
    This ensures that text placed over a heatmap remains legible regardless
    of how dark or light the heatmap cell is.

    Args:
        cmap: A matplotlib colormap object.
        nv (float): The normalized value (0.0 to 1.0) determining the color.

    Returns:
        str: "black" or "white"
    """
    if nv is None or (isinstance(nv, float) and np.isnan(nv)):
        return FG

    rgba = cmap(float(nv))
    # Standard relative luminance formula
    lum = 0.299 * rgba[0] + 0.587 * rgba[1] + 0.114 * rgba[2]

    return "black" if lum > 0.50 else "white"

# =====
# REUSABLE UI WIDGET COMPONENTS
# =====

def page_header(p, title, sub=""):
    """
    Create a standardized, highly styled header block for a dashboard page.
    This maintains visual consistency across all notebook tabs.

    Args:
        p (tk.Widget): Parent widget.
        title (str): Main page title (large, bold).
        sub (str): Optional subtitle/description (smaller, muted).
    """
    hf = tk.Frame(p, bg=CARD, highlightbackground=ACCENT, highlightthickness=2)
    hf.pack(fill="x", padx=18, pady=(12, 6))

    tk.Label(
        hf, text=title, bg=CARD, fg=ACCENT2, font=("Segoe UI", 18, "bold")
    ).pack(anchor="w", padx=16, pady=(10, 2))

    if sub:
        tk.Label(
            hf, text=sub, bg=CARD, fg=MUTED, font=("Segoe UI", 10)
        ).pack(anchor="w", padx=16, pady=(0, 8))

```

```

def scrollable_table(parent, hdrs, cwidths, rows_data, row_colors_fn):
    """
    Constructs a fully custom scrollable data table utilizing a Canvas
    to manage scroll regions containing grid-aligned labels.

    Args:
        parent (tk.Widget): Parent tkinter widget.
        hdrs (list): List of column header strings.
        cwidths (list): List of column widths (in characters).
        rows_data (list of lists): The actual data to display in the rows.
        row_colors_fn (callable): A function that takes (row_idx, col_idx, value)
            and returns a hex color string for the text.
    """
    # Outer frame to hold canvas and scrollbar
    fr = tk.Frame(parent, bg=CARD, highlightbackground=ACCENT, highlightthickness=1)
    fr.pack(side="left", fill="both", expand=True, padx=(0, 10))

    # The canvas allows for scrolling content that exceeds window bounds
    cv = tk.Canvas(fr, bg=CARD, highlightthickness=0)
    vs = ttk.Scrollbar(fr, orient="vertical", command=cv.yview)

    vs.pack(side="right", fill="y")
    cv.pack(side="left", fill="both", expand=True)
    cv.configure(yscrollcommand=vs.set)

    # Inner frame that actually contains the grid of labels
    inner = tk.Frame(cv, bg=CARD)
    win = cv.create_window((0, 0), window=inner, anchor="nw")

    # Dynamic resizing bindings
    inner.bind("<Configure>", lambda e: cv.configure(scrollregion=cv.bbox("all")))
    cv.bind("<Configure>", lambda e: cv.itemconfig(win, width=max(e.width, inner.winfo_reqwidth())))

    # Mouse wheel scrolling binding
    def _on_mousewheel(event):
        # 120 is the standard scroll delta factor on Windows
        cv.yview_scroll(int(-1*(event.delta/120)), "units")

    cv.bind_all("<MouseWheel>", _on_mousewheel)

    # -----
    # Render Table Headers
    # -----
    for i, (h, w) in enumerate(zip(hdrs, cwidths)):
        lbl = tk.Label(
            inner, text=h, bg=BORDER, fg=ACCENT,
            font=FONT_THEAD, width=w, anchor="w"
        )
        lbl.grid(row=0, column=i, padx=8, pady=10, sticky="we")

    # -----
    # Render Table Rows Data
    # -----
    for ridx, vals in enumerate(rows_data, 1):
        # Alternating row colors for better readability (zebra striping)
        cbg = CARD2 if ridx % 2 == 0 else CARD

        for cidx, (val, w) in enumerate(zip(vals, cwidths)):
            col = row_colors_fn(ridx, cidx, val)
            lbl = tk.Label(
                inner, text=val, bg=cbg, fg=col,
                font=FONT_TABLE, width=w, anchor="w"
            )

```



```

        lbl.grid(row=ridx, column=cidx, padx=8, pady=7, sticky="we")

# =====
# DASHBOARD PAGES IMPLEMENTATION
# =====
"""
Builds the Overview Page (Page 1)

This page acts as the executive summary of the entire dashboard.
It displays the most critical Key Performance Indicators (KPIs) on the left
and provides a high-level visual breakdown of platform reach, category
distribution, top hashtags, and average engagement rates on the right.

Args:
    nb (ttk.Notebook): The parent notebook to attach this page to.
    df (pd.DataFrame): The master dataset.
"""
p = tk.Frame(nb, bg=BG)
nb.add(p, text=" Overview ")

body = tk.Frame(p, bg=BG)
body.pack(fill="both", expand=True)

# -----
# Left Panel: KPIs (Key Performance Indicators)
# -----
left = tk.Frame(body, bg=BG, width=300)
left.pack(side="left", fill="y", padx=(8, 6), pady=8)
left.pack_propagate(False) # Prevent frame from shrinking to fit content

# Safely calculate aggregate metrics
try:
    top_platform = df.groupby("platform")["views"].sum().idxmax()
except Exception:
    top_platform = "N/A"

try:
    top_category = df.groupby("category")["engagement_rate"].mean().idxmax()
except Exception:
    top_category = "N/A"

avg_eng = df['engagement_rate'].mean() if not df.empty else 0.0

kpis = [
    ("Total Posts", fmt_num(df["posts"].sum()), "All platforms combined", ACCENT),
    ("Total Views", fmt_num(df["views"].sum()), "Combined reach", ACCENT2),
    ("Total Likes", fmt_num(df["likes"].sum()), "User reactions", GOOD),
    ("Avg Engagement", f"{avg_eng:.2f}%", "(Likes+Cmts+Shares)/Views", MID),
    ("Top Platform", top_platform, "By total views", PLATFORM_COLORS.get(top_platform, FG)),
    ("Trending Category", top_category, "Highest avg engagement", WARN),
]

# Render KPI Cards
for title, val, sub, c in kpis:
    card = tk.Frame(left, bg=CARD, padx=12, pady=8, highlightbackground=BORDER,
highlightthickness=1)
    card.pack(fill="x", pady=3)

    tk.Label(card, text=title, bg=CARD, fg=MUTED, font=("Segoe UI", 10)).pack(anchor="w")
    tk.Label(card, text=val, bg=CARD, fg=c, font=("Segoe UI", 15, "bold")).pack(anchor="w")

    if sub:
        tk.Label(card, text=sub, bg=CARD, fg=BORDER, font=("Segoe UI", 8)).pack(anchor="w")

```

```

# -----
# Right Panel: Visualization Grid
# -----

right = tk.Frame(body, bg=BG)
right.pack(side="left", fill="both", expand=True, padx=(8, 18), pady=8)

# Setup 2x2 Grid using GridSpec
fig = Figure(figsize=(10.5, 5.5), dpi=DPI)
fig.patch.set_facecolor(BG)
gs = GridSpec(2, 2, figure=fig, hspace=0.50, wspace=0.40)

# --- Chart 1: Platform Reach (Horizontal Bar) ---
ax1 = fig.add_subplot(gs[0, 0])
if not df.empty:
    pv = df.groupby("platform")["views"].sum().sort_values()
    hbar(ax1, pv.index, pv.values/1e6, [PLATFORM_COLORS.get(p, ACCENT) for p in pv.index], fs=9,
bh=0.55, vlabels=False)
    style_ax(ax1, "Platform Reach (Million Views)")

# --- Chart 2: Category Distribution (Donut Pie Chart) ---
ax2 = fig.add_subplot(gs[1, 0])
if not df.empty:
    cat_cnt = df["category"].value_counts()
    clr = [CATEGORY_COLORS.get(c, ACCENT) for c in cat_cnt.index]

    wedges, _, autotexts = ax2.pie(
        cat_cnt.values, colors=clr,
        autopct=lambda p: f"{p:.0f}%" if p >= 5 else "",
        pctdistance=0.75, startangle=90,
        wedgeprops=dict(width=0.45, edgecolor=BG, linewidth=2.5),
        radius=0.80
    )

    # Adjust text color inside pie slices for readability
    for at, wdg in zip(autotexts, wedges):
        fc = wdg.get_facecolor()
        lum = 0.299*fc[0] + 0.587*fc[1] + 0.114*fc[2]
        at.set_fontsize(8)
        at.set_fontweight("bold")
        at.set_color("black" if lum > 0.5 else "white")

    ax2.legend(wedges, [f"{c}({n})" for c, n in cat_cnt.items()],
        loc="upper center", bbox_to_anchor=(0.5, 0.0), ncol=2,
        fontsize=7, labelcolor=FG, facecolor=CARD, edgecolor=BORDER)

ax2.set_title("Category Distribution", color=ACCENT2, fontsize=11, fontweight="bold", pad=8)

# --- Chart 3: Top Hashtags (Horizontal Bar) ---
ax3 = fig.add_subplot(gs[0, 1])
if not df.empty:
    ht = df.groupby("hashtag")["views"].sum().nlargest(5).sort_values()
    hbar(ax3, ht.index, ht.values/1e6, [ACCENT]*5, fs=9, bh=0.6)
    style_ax(ax3, "Top 5 Hashtags (M Views)")

# --- Chart 4: Average Engagement (Horizontal Bar with Avg Line) ---
ax4 = fig.add_subplot(gs[1, 1])
if not df.empty:
    pe = df.groupby("platform")["engagement_rate"].mean().sort_values()
    hbar(ax4, pe.index, pe.values, [PLATFORM_COLORS.get(p, ACCENT) for p in pe.index], fs=9,
bh=0.55)

    # Add benchmark line at 3% engagement
    ax4.axvline(3, color=MID, lw=1.3, ls="--", label="Avg 3%")

```

```

        ax4.legend(fontsize=8, labelcolor=FG, facecolor=CARD, edgecolor=BORDER)

    style_ax(ax4, "Avg Engagement Rate (%)")

    # Layout adjustment and embedding
    try:
        fig.tight_layout(pad=1.5)
    except Exception as e:
        pass

    embed(fig, right)

def page2(nb, df):
    """
    Builds the Platform Analysis Page (Page 2)

    This page isolates platform performance. It provides a detailed data table
    containing all base metrics grouped by platform, accompanied by a 2x2 grid
    of standard vertical bar charts comparing Likes, Shares, Comments, and Engagement.

    Args:
        nb (ttk.Notebook): The parent notebook.
        df (pd.DataFrame): The master dataset.
    """
    p = tk.Frame(nb, bg=BG)
    nb.add(p, text=" Platform Analysis ")

    page_header(p, "Platform-wise Social Media Performance",
                "Detailed metrics comparison across all 5 platforms")

    body = tk.Frame(p, bg=BG)
    body.pack(fill="both", expand=True, padx=16, pady=6)

    if df.empty:
        return

    # Aggregate data by platform
    plat_df = df.groupby("platform").agg(
        posts=("posts", "sum"),
        likes=("likes", "sum"),
        views=("views", "sum"),
        comments=("comments", "sum"),
        shares=("shares", "sum"),
        avg_eng=("engagement_rate", "mean")
    ).reset_index().sort_values("views", ascending=False)

    # -----
    # Left Component: Data Table
    # -----
    hdrs = ["PLATFORM", "POSTS", "LIKES", "VIEWS", "COMMENTS", "SHARES", "ENG%"]
    cw = [14, 11, 11, 12, 11, 11, 9]

    rows = [
        [
            r["platform"],
            fmt_num(r["posts"]),
            fmt_num(r["likes"]),
            fmt_num(r["views"]),
            fmt_num(r["comments"]),
            fmt_num(r["shares"]),
            f"{r['avg_eng']:.2f}%"
        ]
    ]
    for _, r in plat_df.iterrows()

```

```

]

# Color logic: Platform names are colored, metrics remain default foreground
def colorfn(ri, ci, val):
    return PLATFORM_COLORS.get(str(val), FG) if ci == 0 else FG

scrollable_table(body, hdrs, cw, rows, colorfn)

# -----
# Right Component: Multi-metric Bar Charts
# -----
ch = tk.Frame(body, bg=BG)
ch.pack(side="left", fill="both", expand=True)

fig = Figure(figsize=(8.5, 6.0), dpi=DPI)
fig.patch.set_facecolor(BG)
gs = GridSpec(2, 2, figure=fig, hspace=0.65, wspace=0.46)

plats = plat_df["platform"]
clrs = [PLATFORM_COLORS.get(p, ACCENT) for p in plats]

# Define the 4 metrics to plot in the 2x2 grid
metrics = [
    ("likes", "Likes (M)"),
    ("shares", "Shares (M)"),
    ("comments", "Comments (M)"),
    ("avg_eng", "Avg Eng %")
]

for i, (col, title) in enumerate(metrics):
    ax = fig.add_subplot(gs[i//2, i%2])
    # Convert absolute metrics to millions for cleaner y-axis
    vals = plat_df[col]/1e6 if col != "avg_eng" else plat_df[col]

    ax.bar(plats, vals, color=clrs, edgecolor=BORDER, linewidth=0.5)
    style_ax(ax, title)
    ax.tick_params(axis="x", rotation=15, colors=FG)

fig.subplots_adjust(left=0.12, right=0.97, top=0.94, bottom=0.12)
embed(fig, ch)

def page3(nb, df):
    """
    Builds the Category Trends Page (Page 3)

    This page analyzes data by the content category (e.g., Sports, Entertainment).
    It features views vs engagement analysis, top hashtags within categories,
    and the distribution of content output.

    Args:
        nb (ttk.Notebook): The parent notebook.
        df (pd.DataFrame): The master dataset.
    """
    p = tk.Frame(nb, bg=BG)
    nb.add(p, text="  Category Trends  ")

    page_header(p, "Content Category Performance Analysis",
                "Engagement and reach metrics by content category")

    if df.empty:
        return

    fig = Figure(figsize=(12, 6.5), dpi=DPI)

```

```

fig.patch.set_facecolor(BG)
gs = GridSpec(2, 2, figure=fig, hspace=0.50, wspace=0.35)

# Aggregate by Category
cat_df = df.groupby("category").agg(
    views=("views", "sum"),
    eng=("engagement_rate", "mean")
).sort_values("views", ascending=False)

# --- Top Left: Views by Category ---
ax1 = fig.add_subplot(gs[0, 0])
ax1.barh(
    cat_df.index, cat_df["views"]/1e6,
    color=[CATEGORY_COLORS.get(c, ACCENT) for c in cat_df.index],
    height=0.6, edgecolor=BORDER, linewidth=0.5
)
style_ax(ax1, "Views by Category (M)")
fix_yticks(ax1, 9)

# --- Top Right: Engagement by Category ---
ax2 = fig.add_subplot(gs[0, 1])
ce = cat_df.sort_values("eng")
ax2.barh(
    ce.index, ce["eng"],
    color=[CATEGORY_COLORS.get(c, ACCENT) for c in ce.index],
    height=0.6, edgecolor=BORDER, linewidth=0.5
)
ax2.axvline(3, color=MID, lw=1.3, ls="--")
style_ax(ax2, "Avg Engagement Rate (%)")
fix_yticks(ax2, 9)

# --- Bottom Left: Top 10 Hashtags by Engagement ---
ax3 = fig.add_subplot(gs[1, 0])
ht = df.groupby("hashtag")["engagement_rate"].mean().nlargest(10).sort_values()
ax3.barh(
    ht.index, ht.values,
    color=[ACCENT2]*10,
    height=0.6, edgecolor=BORDER, linewidth=0.5
)
style_ax(ax3, "Top 10 Hashtags by Eng Rate")
fix_yticks(ax3, 8)

# --- Bottom Right: Posts Output per Category (Pie) ---
ax4 = fig.add_subplot(gs[1, 1])
cat_posts = df["category"].value_counts()
clrs = [CATEGORY_COLORS.get(c, ACCENT) for c in cat_posts.index]

wedges, _, ats = ax4.pie(
    cat_posts.values, colors=clrs,
    autopct=lambda p: f"{p:.0f}%" if p >= 5 else "",
    pctdistance=0.75, startangle=90,
    wedgeprops=dict(width=0.45, edgecolor=BG, linewidth=2),
    radius=0.80
)

for at, wdg in zip(ats, wedges):
    fc = wdg.get_facecolor()
    lum = 0.299*fc[0] + 0.587*fc[1] + 0.114*fc[2]
    at.set_fontsize(8)
    at.set_fontweight("bold")
    at.set_color("black" if lum > 0.5 else "white")

ax4.set_title("Posts per Category", color=ACCENT2, fontsize=11, fontweight="bold", pad=8)
ax4.legend(

```

```

        wedges, cat_posts.index,
        loc="upper center", bbox_to_anchor=(0.5, 0.0),
        ncol=2, fontsize=7, labelcolor=FG,
        facecolor=CARD, edgecolor=BORDER
    )

fig.subplots_adjust(left=0.18, right=0.96, top=0.92, bottom=0.08)
embed(fig, p)

def page4(nb, df):
    """
    Builds the Hashtag Rankings Page (Page 4)

    This page is designed for deep-diving into specific topics (hashtags).
    It features a comprehensive scrollable ranking table alongside
    two charts highlighting the absolute best and worst performers.

    Args:
        nb (ttk.Notebook): The parent notebook.
        df (pd.DataFrame): The master dataset.
    """
    p = tk.Frame(nb, bg=BG)
    nb.add(p, text=" Hashtag Rankings ")

    page_header(p, "Hashtag Performance Rankings",
                "Top and bottom performing hashtags by total views")

    outer = tk.Frame(p, bg=BG)
    outer.pack(fill="both", expand=True, padx=16, pady=6)

    if df.empty:
        return

    # Aggregate data by hashtag, taking the first category (assuming 1-to-1 mapping)
    ht_df = df.groupby("hashtag").agg(
        category=("category", "first"),
        views=("views", "sum"),
        likes=("likes", "sum"),
        eng=("engagement_rate", "mean"),
        posts=("posts", "sum")
    ).reset_index().sort_values("views", ascending=False).reset_index(drop=True)

    # -----
    # Left Component: Full Rankings Table
    # -----
    hdrs = ["#", "HASHTAG", "CATEGORY", "VIEWS", "LIKES", "ENG%", "POSTS"]
    cw = [4, 16, 14, 11, 11, 9, 11]

    rows = [
        [
            i+1,
            r["hashtag"],
            r["category"],
            fmt_num(r["views"]),
            fmt_num(r["likes"]),
            f"{r['eng']:.2f}%",
            fmt_num(r["posts"])
        ]
        for i, (_, r) in enumerate(ht_df.iterrows())
    ]

    def colorfn(ri, ci, val):
        if ci == 0: return MUTED

```

```

        if ci == 2: return CATEGORY_COLORS.get(str(val), FG)
        return FG

scrollable_table(outer, hdrs, cw, rows, colorfn)

# -----
# Right Component: Top & Bottom 10 Visualizations
# -----
frr = tk.Frame(outer, bg=CARD, highlightbackground=ACCENT, highlightthickness=1)
frr.pack(side="left", fill="both", expand=True)

fig = Figure(figsize=(6.8, 6.0), dpi=DPI)
fig.patch.set_facecolor(CARD)

# Top 10 Performers
ax1 = fig.add_subplot(211)
ax1.set_facecolor(CARD)
t10 = ht_df.head(10).sort_values("views")
hbar(ax1, t10["hashtag"], t10["views"]/1e6, [WARN]*10, fs=9, bh=0.60, vlabels=False)
ax1.set_title("[HIGH] Top 10 Most Viewed", color=ACCENT2, fontsize=10, fontweight="bold", pad=6)
fix_yticks(ax1, 9)

# Bottom 10 Performers
ax2 = fig.add_subplot(212)
ax2.set_facecolor(CARD)
b10 = ht_df.tail(10).sort_values("views", ascending=False)
hbar(ax2, b10["hashtag"], b10["views"]/1e6, [GOOD]*10, fs=9, bh=0.60, vlabels=False)
ax2.set_title("[LOW] Bottom 10 Least Viewed", color=ACCENT2, fontsize=10, fontweight="bold", pad=6)
fix_yticks(ax2, 9)

fig.subplots_adjust(left=0.32, right=0.96, top=0.92, bottom=0.08, hspace=0.60)
embed(fig, frr)

def page5(nb, df):
    """
    Builds the Heatmap & Correlation Page (Page 5)

    This page focuses on statistical relationships.
    1. A Correlation Matrix showing how numerical metrics relate.
    2. A 2D Heatmap showing Platform vs Category engagement intensity.

    Args:
        nb (ttk.Notebook): The parent notebook.
        df (pd.DataFrame): The master dataset.
    """
    p = tk.Frame(nb, bg=BG)
    nb.add(p, text=" Heatmap & Correlation ")

    page_header(p, "Metric Correlation & Platform x Category Heatmap",
                "Left: inter-metric correlation | Right: normalised engagement heatmap")

    if df.empty:
        return

    fig = Figure(figsize=(12, 5.8), dpi=DPI)
    fig.subplots_adjust(left=0.10, right=0.96, top=0.88, bottom=0.18, wspace=0.35)
    fig.patch.set_facecolor(BG)

    # --- Left: Metric Correlation Matrix ---
    cols = ["posts", "likes", "views", "comments", "shares", "engagement_rate"]
    lbls = ["Posts", "Likes", "Views", "Comments", "Shares", "Eng Rate"]

    # Calculate Pearson correlation coefficient

```

```
corr = df[cols].corr()

ax1 = fig.add_subplot(121)
im1 = ax1.imshow(corr.values, cmap="PuOr", vmin=-1, vmax=1, aspect="auto")

# Set up ticks and labels
ax1.set_xticks(range(len(lbls)))
ax1.set_xticklabels(lbls, rotation=35, ha="right", fontsize=10, color=FG)
ax1.set_yticks(range(len(lbls)))
ax1.set_yticklabels(lbls, fontsize=10, color=FG)

# Overlay correlation values text onto the matrix cells
cmap1 = plt.get_cmap("PuOr")
for i in range(len(lbls)):
    for j in range(len(lbls)):
        v = corr.values[i, j]
        # Calculate appropriate text color based on cell background
        tc = cell_tc(cmap1, (v+1)/2)
        ax1.text(j, i, f"{v:.2f}", ha="center", va="center",
                fontsize=10, fontweight="bold", color=tc)

cb1 = fig.colorbar(im1, ax=ax1, shrink=0.80)
cb1.ax.yaxis.set_tick_params(color=FG, labels=10)
plt.setp(cb1.ax.yaxis.get_ticklabels(), color=FG)
style_ax(ax1, "Metric Correlation Matrix")

# --- Right: Platform x Category Heatmap ---
# Create pivot table for 2D representation
pivot = df.pivot_table(index="platform", columns="category", values="engagement_rate",
aggfunc="mean")

# Normalize data using Min-Max scaling for visual colormapping
norm = (pivot - pivot.min()) / (pivot.max() - pivot.min())

ax2 = fig.add_subplot(122)
cmap2 = plt.get_cmap("magma")
im2 = ax2.imshow(norm.values, cmap="magma", aspect="auto", vmin=0, vmax=1)

# Setup axis ticks
ax2.set_xticks(range(len(pivot.columns)))
ax2.set_xticklabels([trunc(c, 9) for c in pivot.columns], fontsize=9, color=FG, rotation=35,
ha="right")
ax2.set_yticks(range(len(pivot.index)))
ax2.set_yticklabels(pivot.index, fontsize=10, color=FG)

# Overlay engagement rate values text
for i in range(len(pivot.index)):
    for j in range(len(pivot.columns)):
        rv = pivot.values[i, j]
        nv = norm.values[i, j]

        # Handle empty cells (NaN) gracefully
        txt = "nan" if (isinstance(rv, float) and np.isnan(rv)) else f"{rv:.1f}"
        tc = cell_tc(cmap2, None if (isinstance(nv, float) and np.isnan(nv)) else nv)

        ax2.text(j, i, txt, ha="center", va="center", fontsize=9, color=tc)

cb2 = fig.colorbar(im2, ax=ax2, shrink=0.80)
cb2.ax.yaxis.set_tick_params(color=FG, labels=10)
plt.setp(cb2.ax.yaxis.get_ticklabels(), color=FG)
style_ax(ax2, "Platform x Category Engagement")

embed(fig, p)
```



```

# =====
# MAIN APPLICATION ASSEMBLY AND LAUNCH
# =====

def build_dashboard(root, df):
    """
    Assemble the complete dashboard structure including the main notebook and all pages.
    This acts as the orchestrator combining all previous UI functions.

    Args:
        root (tk.Tk): Main application window.
        df (pd.DataFrame): Dataset to visualize.
    """

    # --- Top Application Header ---
    hdr = tk.Frame(root, bg=CARD, highlightbackground=ACCENT, highlightthickness=2)
    hdr.pack(fill="x", padx=16, pady=(10, 4))

    lh = tk.Frame(hdr, bg=CARD)
    lh.pack(side="left", padx=14, pady=10)
    tk.Label(
        lh, text="Social Media Trends Analytics Dashboard",
        bg=CARD, fg=FG, font=("Segoe UI", 22, "bold")
    ).pack(anchor="w")

    # Prepare dynamic subtext based on dataset dimensions
    try:
        stats_str = (
            f"{len(df)} Records | "
            f"{df['platform'].nunique()} Platforms | "
            f"{df['category'].nunique()} Categories | "
            f"{df['country'].nunique()} Countries"
        )
    except Exception:
        stats_str = "Data loading error - using default schema"

    tk.Label(lh, bg=CARD, fg=MUTED, font=("Segoe UI", 10), text=stats_str).pack(anchor="w")

    # Right side author block
    rh = tk.Frame(hdr, bg=CARD)
    rh.pack(side="right", padx=14, pady=10)

    tk.Label(rh, text="DVA Assignment 2", bg=CARD, fg=ACCENT, font=("Segoe UI", 12,
"bold")).pack(anchor="e")
    tk.Label(rh, text="Akshat Tripathi | 04914202023", bg=CARD, fg=ACCENT2, font=("Segoe UI", 10,
"bold")).pack(anchor="e")
    tk.Label(rh, text="IPU BCA 6th Semester", bg=CARD, fg=MUTED, font=("Segoe UI", 9)).pack(anchor="e")

    # --- Notebook (Tabbed Interface) Setup ---
    st = ttk.Style()
    st.theme_use("default")

    # Configure colors and fonts for notebook tabs to match the dark theme
    st.configure("TNotebook", background=BG, borderwidth=0)
    st.configure("TNotebook.Tab", background=CARD, foreground=FG, padding=(28, 11), font=("Segoe UI",
11, "bold"))
    st.map("TNotebook.Tab", background=[("selected", "#B983FF")], foreground=[("selected", "#13111C")])

    nb = ttk.Notebook(root, style="TNotebook")
    nb.pack(fill="both", expand=True, padx=12, pady=8)

    # Initialize and append all 5 analytical pages
    pagel(nb, df)

```

```
page2(nb, df)
page3(nb, df)
page4(nb, df)
page5(nb, df)

def main():
    """
    Application entry point.
    Initializes the Tkinter main loop, configures the base window,
    loads the data, and triggers the dashboard build process.
    """

    # Initialize main window
    root = tk.Tk()
    root.title("Social Media Trends Dashboard - Akshat Tripathi | 04914202023")
    root.configure(bg=BG)

    # Attempt to maximize window based on OS capabilities
    try:
        root.state("zoomed") # Works on Windows
    except tk.TclError:
        try:
            root.attributes('-zoomed', True) # Works on some Linux environments
        except Exception as e:

    # Load dataset
    df = load_data()

    # Construct UI
    build_dashboard(root, df)

    # Start application event loop
    root.mainloop()

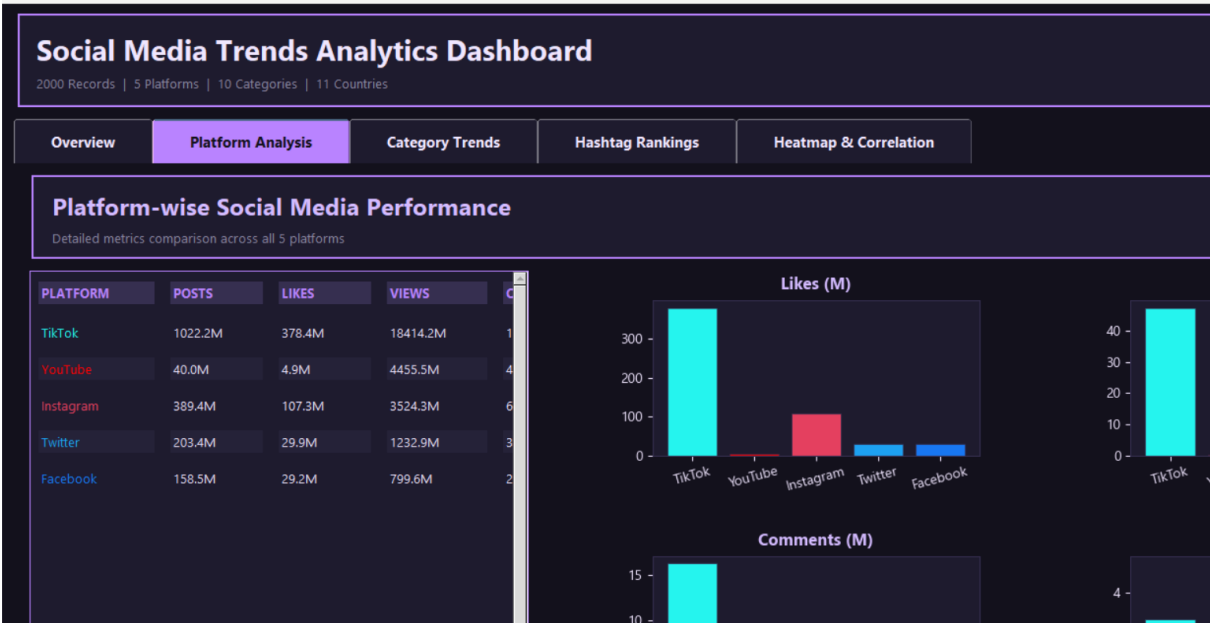
if __name__ == "__main__":
    # Execute main function only when script is run directly
    main()
```

Output

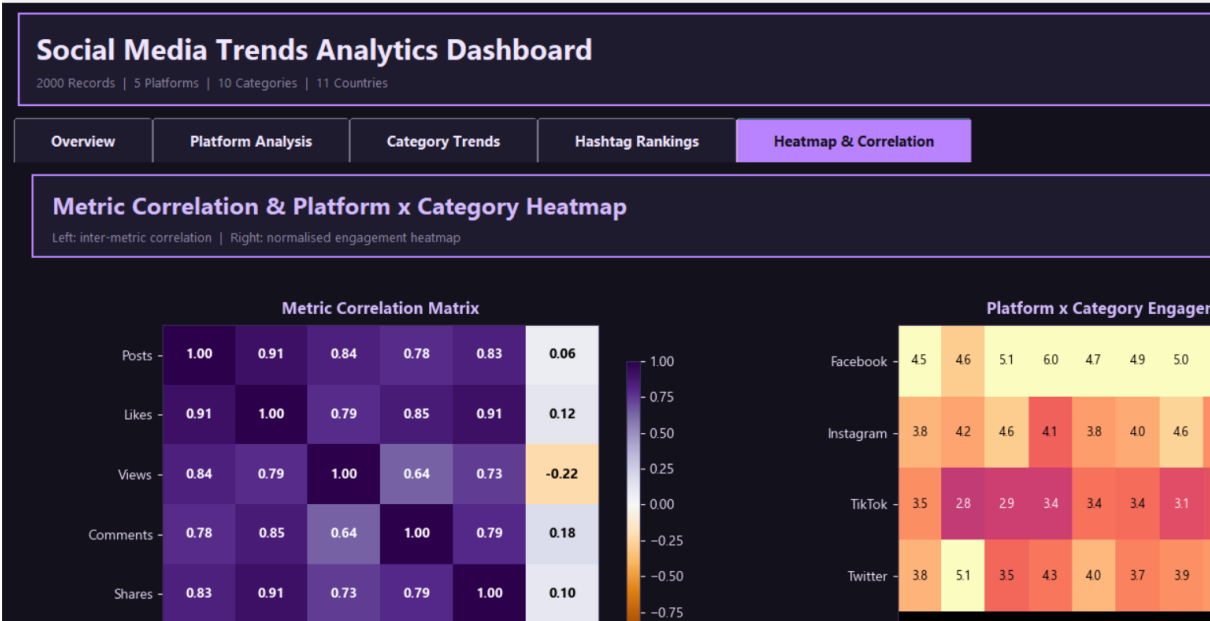
Page 1- Overview



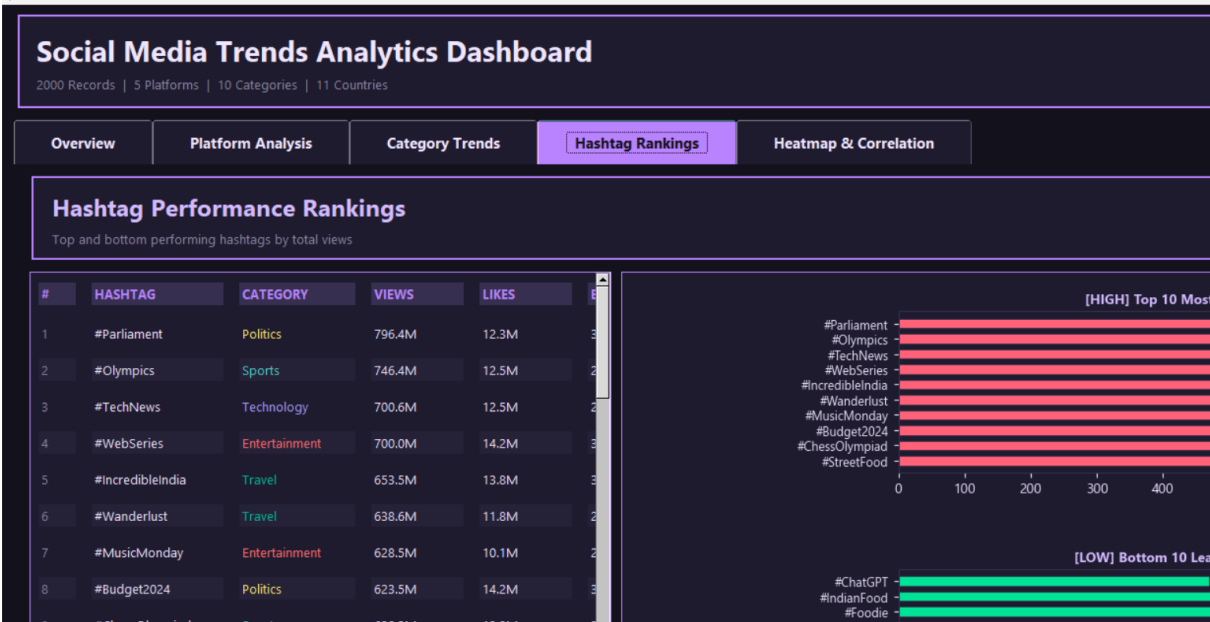
Page 2- Platform Analysis



Page 3- Category Trends



Page 4- Hashtag Rankings



Page 5- Heatmap & Correlations

